Project name: University system

Discipline: Object-Oriented programming

Team members:
Shakhar Yelnur
Yer-Sultan Zhuzzhigit
Shakhar Yernur

The main goal of the project:

In this project, we have implemented a research-oriented university system that runs on the console. Our program is an ideal place for students and teachers at the university, as it provides everything they need in one system. Students can easily register for courses, view their marks and transcript, rate teachers, join student organizations, and subscribe to research journals. Teachers, in turn, can manage their courses, put marks, send complaints to the dean, publish research papers, and calculate their h-index. This makes the system a convenient and centralized tool for handling both academic and research activities within the university.

Project description:

1. Create an architecture using Use Case and UML class Diagrams.

2. Writing code in Eclipse

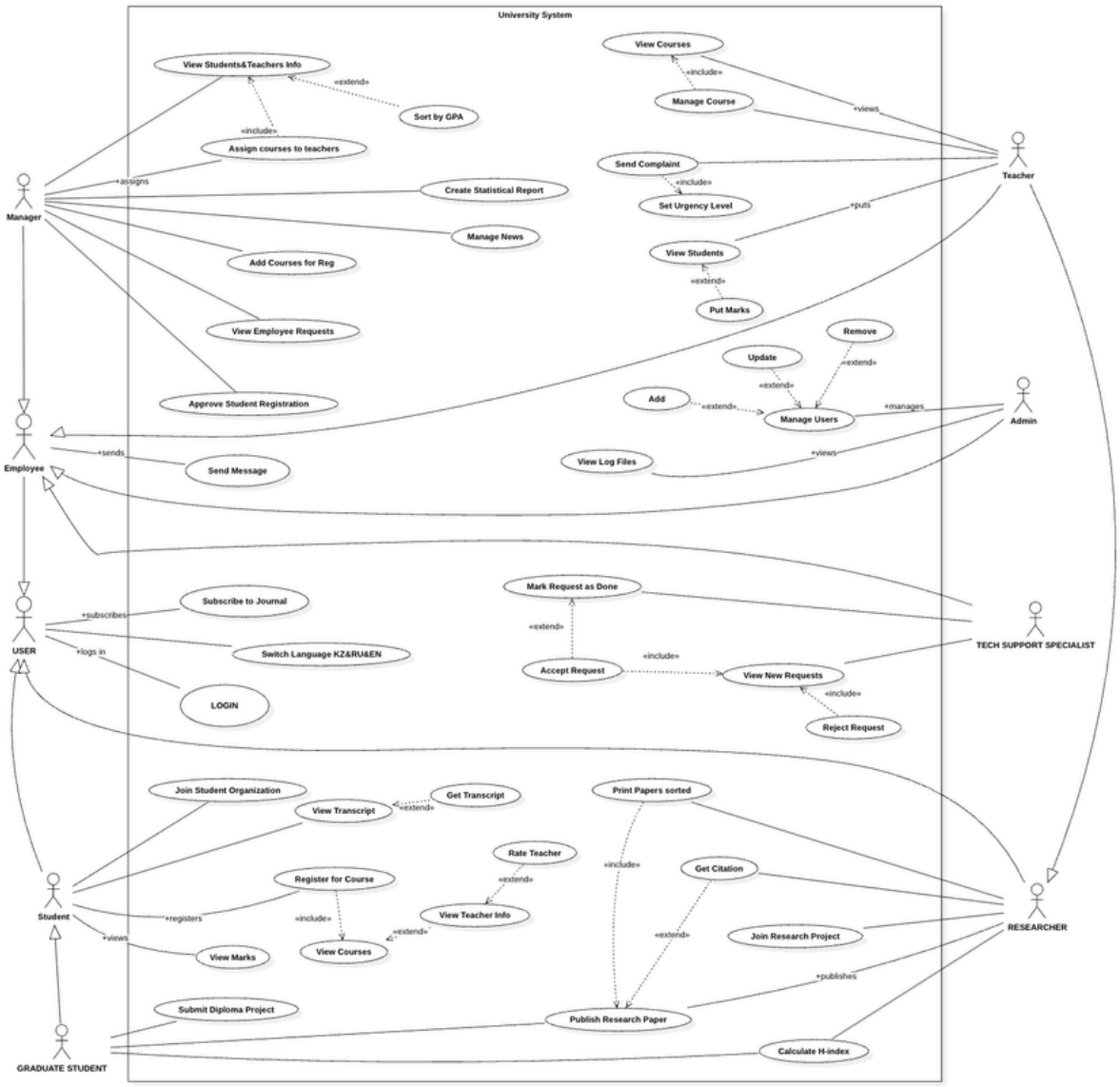
3. Create Report and Presentation

1. Diagrams:

We created diagrams using the StarUML tool. The UML is divided into two parts:

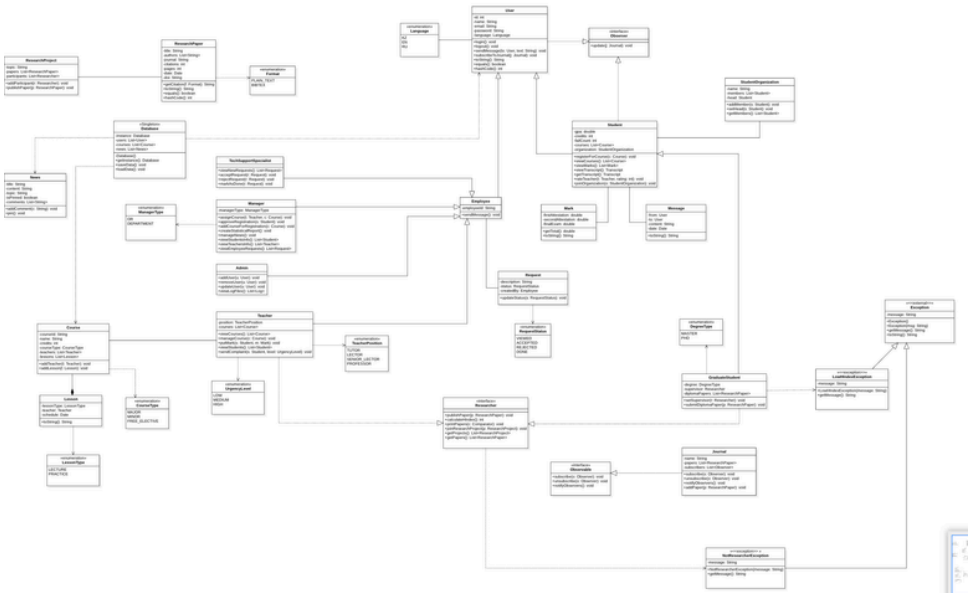
1) Use-case diagram
2) Class diagram

Use-Case diagram:



Here we have 9 actors – User, Employee, Student, Teacher, Manager, Admin, Tech Support Specialist, Researcher, Graduate Student

Class-Diagram:



2.Writing code in Eclipse :

Packages:

We have divided the classes into 6 main packages:

- models — main actors and entities: User, Student, Teacher, Manager, Admin, Course, Mark, Lesson, News, ResearchPaper, ResearchProject, etc.
- interfaces — core abstractions: Researcher, Observable, Observer
- enums — all enumerations: TeacherPosition, CourseType, UrgencyLevel, Language, etc.
- exceptions — custom exceptions: LowHIndexException, NotResearcherException
- factory — object creation: UserFactory, JournalFactory
- database — data storage and persistence: Database
- util — utility classes: Translator (KZ, EN, RU)

The User class has such fields as:

```
public abstract class User implements Observer, Serializable, Comparable<User> {
    private static final long serialVersionUID = 1L;

    protected int id;
    protected String name;
    protected String email;
    protected String password;
    protected Language lang;
    protected List<Message> inbox = new ArrayList<>();
```

Student is a subclass of the User and the Student class itself has fields such as:

```
public class Student extends User {
    private static final long serialVersionUID = 1L;

    public static final int MAX_CREDITS = 21;
    public static final int MAX_FAILS = 3;

    protected double gpa;
    protected int credits;
    protected int failCount;
    protected String major;
    protected String school;
    protected int year;
    protected List<Course> courses = new ArrayList<>();

    protected Map<Course, Mark> marks = new LinkedHashMap<>();

    protected StudentOrganization organization;

    protected Map<Teacher, Integer> ratings = new HashMap<>();
```

registerForCourse() method:

```
public boolean registerForCourse(Course c) {
    if (failCount >= MAX_FAILS) {
        System.out.println("ERROR: " + name + " has failed " + failCount + " times, cannot register.");
        return false;
    }
    if (credits + c.getCredits() > MAX_CREDITS) {
        System.out.println("ERROR: " + name + " cannot register for " + c.getName() + " - credit limit ("
                + MAX_CREDITS + ") exceeded.");
        return false;
    }
    if (c.getType() == CourseType.MAJOR && !canTakeAsMajor(c)) {
        System.out.println("ERROR: " + c.getName() + " is not in " + name + "'s major.");
        return false;
    }
    courses.add(c);
    c.addStudent(this);
    credits += c.getCredits();
    marks.put(c, new Mark());
    System.out.println("OK: " + name + " registered for " + c.getName());
    return true;
}
```

A student can view detailed information about the teachers of a specific course they are enrolled in. The system first shows the list of the student's registered courses, then displays all teachers assigned to the selected course along with their h-index, and also shows the lessons (lecture/practice) with room information.
In Main.lava class
handleStudent() method:

```
 case "9":
     Course coT = pickCourseFromList(s.viewCourses());
     if (coT == null)
         return;
     System.out.println("Teachers of " + coT.getName() + ":");
     for (Teacher t9 : coT.getTeachers()) {
         System.out.println("  - " + t9 + " | h-index=" + t9.calculateHIndex());
     }
     System.out.println("Lessons:");
     for (Lesson l : coT.getLessons())
         System.out.println("  - " + l);
     break;

 }
```

And Student can rate Teacher :

```
public void rateTeacher(Teacher t, int rating) {
    if (rating < 1 || rating > 5) {
        System.out.println("Rating must be 1-5");
        return;
    }
    ratings.put(t, rating);
    System.out.println(name + " rated " + t.getName() + " with " + rating);
}
```

Student mode:
The handleStudent() method in Main.java processes all student actions. Our student menu includes course registration with credit and fail count validation, transcript viewing with GPA calculation, teacher rating, joining student organizations, and viewing teacher information with h-index for each course.

```
private static void handleStudent(String c) {
    Student s = (Student) current;
    switch (c) {
    case "1":
        for (Course co : db.getCourses())
            System.out.println(co);
        break;
    case "2":
        Course co = pickCourse();
        if (co != null)
            s.registerForCourse(co);
        break;
    case "3":
        for (Course x : s.viewCourses())
            System.out.println(x);
        break;
    case "4":
        for (Map.Entry<Course, Mark> e : s.viewMarks().entrySet())
            System.out.println(e.getKey().getName() + " -> " + e.getValue());
        break;
    case "5":
        System.out.println(s.getTranscript());
        break;
    case "6":
        Teacher t = pickTeacher();
        if (t == null)
            return;
        System.out.print("Rating (1-5): ");
        int rt = Integer.parseInt(sc.nextLine().trim());
        s.rateTeacher(t, rt);
        break;
    case "7":
        List<StudentOrganization> os = db.getOrgs();
        if (os.isEmpty()) {
            System.out.println("No organizations");
            return;
        }
        for (int i = 0; i < os.size(); i++)
            System.out.println((i + 1) + ") " + os.get(i));
        System.out.print("Choose: ");
        int oi = Integer.parseInt(sc.nextLine().trim()) - 1;
        s.joinOrganization(os.get(oi));
        break;
    case "8":
        User to = pickUser();
        if (to == null)
            return;
        System.out.print("Text: ");
        String tx = sc.nextLine().trim();
        s.sendMessage(to, tx);
        break;
    case "9":
        Course coT = pickCourseFromList(s.viewCourses());
        if (coT == null)
            return;
        System.out.println("Teachers of " + coT.getName() + ":");
        for (Teacher t9 : coT.getTeachers()) {
            System.out.println("  - " + t9 + " | h-index=" + t9.calculateHIndex());
        }
        System.out.println("Lessons:");
        for (Lesson l : coT.getLessons())
            System.out.println("  - " + l);
        break;

    }
}
```

We have class Mark and has fields like:

```
public class Mark implements Serializable, Comparable<Mark> {
    private static final long serialVersionUID = 1L;

    private double firstAtt;
    private double secondAtt;
    private double finalExam;
```

Our Mark class stores three separate grade components: firstAtt, secondAtt, and finalExam as double values. After all getters and setters, we wrote the method getTotal() which calculates the overall grade, and getLetter() which converts the total score to a letter grade. Our implementation also includes Comparable<Mark>, equals(), hashCode(), and toString().

getTotal() and getLetter() methods:

```
public double getTotal() {
    return firstAtt + secondAtt + finalExam;
}
```

```
public String getLetter() {
    double t = getTotal();
    if (t >= 95)
        return "A";
    if (t >= 90)
        return "A-";
    if (t >= 85)
        return "B+";
    if (t >= 80)
        return "B";
    if (t >= 75)
        return "B-";
    if (t >= 70)
        return "C+";
    if (t >= 65)
        return "C";
    if (t >= 60)
        return "C-";
    if (t >= 55)
        return "D+";
    if (t >= 50)
        return "D";
    return "F";
}
```

We have class Teacher and has fields like:

```
public class Teacher extends Employee implements Researcher {
    private static final long serialVersionUID = 1L;

    private TeacherPosition position;
    private List<Course> courses = new ArrayList<>();
    private List<ResearchPaper> papers = new ArrayList<>();
    private List<ResearchProject> projects = new ArrayList<>();
    private String school;
```

putMark() method:

```
public void putMark(Student s, Course c, Mark m) {
    if (!s.getCourses().contains(c)) {
        System.out.println("Student is not registered for this course.");
        return;
    }
    s.receiveMark(c, m);
    System.out.println(name + " put marks for " + s.getName() + " in " + c.getName() + ": " + m);
}
```

sendComplaint() method:

```
public void sendComplaint(Manager dean, Student s, UrgencyLevel level, String reason) {
    Request r = new Request("Complaint about " + s.getName() + " (" + level + "): " + reason, this);
    dean.receiveComplaint(r, level);
    System.out
            .println("[COMPLAINT-" + level + "] " + name + " -> dean " + dean.getName() + " about " + s.getName());
}
```

Also we have interface Researcher:

```
public interface Researcher {
    List<ResearchPaper> getPapers();

    List<ResearchProject> getProjects();

    String getResearcherName();
```

calculateHIndex() method:

```
default int calculateHIndex() {
    List<Integer> cs = getPapers().stream().map(ResearchPaper::getCitations).sorted(Comparator.reverseOrder())
            .collect(Collectors.toList());
    int h = 0;
    for (int i = 0; i < cs.size(); i++) {
        if (cs.get(i) >= i + 1)
            h = i + 1;
        else
            break;
    }
    return h;
}
```

printPapers() method:

```
default void printPapers(Comparator<ResearchPaper> c) {
    List<ResearchPaper> sorted = new ArrayList<>(getPapers());
    sorted.sort(c);
    for (ResearchPaper p : sorted)
        System.out.println("  - " + p);
}
```

getCitation() method:

```
public String getCitation(Format f) {
    SimpleDateFormat sdf = new SimpleDateFormat("yyyy");
    String year = sdf.format(date);
    if (f == Format.PLAIN_TEXT) {
        return String.join(", ", authors) + " (" + year + "). " + title + ". " + journal + ", pp. " + pages
                + ". DOI: " + doi;
    } else {
        String key = (authors.isEmpty() ? "anon" : authors.get(0).split(" ")[0]) + year;
        return "@article{" + key + ",\n" + "  title={" + title + "},\n" + "  author={"
                + String.join(" and ", authors) + "},\n" + "  journal={" + journal + "},\n" + "  year={" + year
                + "},\n" + "  pages={" + pages + "},\n" + "  doi={" + doi + "}\n}";
    }
}
```

Patterns

1. Singleton

```
  private static Database instance;

  private Database() {}

  public static Database getInstance() {
      if (instance == null)
          instance = new Database();
      return instance;
  }
```

2. Serialization (saveData/loadData)

```
public void saveData() {
    new File("data").mkdirs();
    try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(FILE))) {
        oos.writeObject(this);
        System.out.println("[DB] State saved to " + FILE);
    } catch (IOException e) {
        System.out.println("[DB] Save failed: " + e.getMessage());
    }
}

public static void loadData() {
    File f = new File(FILE);
    if (!f.exists()) {
        System.out.println("[DB] No saved state, starting fresh");
        return;
    }
    try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(f))) {
        instance = (Database) ois.readObject();
        System.out.println("[DB] State loaded from " + FILE);
    } catch (IOException | ClassNotFoundException e) {
        System.out.println("[DB] Load failed: " + e.getMessage());
    }
}
```

3.Observer

```
@Override
public void subscribe(Observer o) {
    if (!subscribers.contains(o)) subscribers.add(o);
}

@Override
public void unsubscribe(Observer o) {
    subscribers.remove(o);
}

@Override
public void notifyObservers() {
    for (Observer o : subscribers) o.update(this);
}

public void addPaper(ResearchPaper p) {
    papers.add(p);
    notifyObservers();
}
```

4. Factory

```
public class UserFactory {

    public static User createUser(String role, int id, String name, String email, String pass, Language lang,
            Object... extra) {
        switch (role.toUpperCase()) {
        case "ADMIN":
            return new Admin(id, name, email, pass, lang, (String) extra[0]);
        case "MANAGER":
            return new Manager(id, name, email, pass, lang, (String) extra[0], (ManagerType) extra[1]);
        case "TEACHER":
            return new Teacher(id, name, email, pass, lang, (String) extra[0], (TeacherPosition) extra[1],
                    (String) extra[2]);
        case "STUDENT":
            return new Student(id, name, email, pass, lang, (String) extra[0], (String) extra[1], (Integer) extra[2]);
        case "GRADUATE":
            return new GraduateStudent(id, name, email, pass, lang, (String) extra[0], (String) extra[1],
                    (Integer) extra[2], (DegreeType) extra[3]);
        case "SUPPORT":
            return new TechSupportSpecialist(id, name, email, pass, lang, (String) extra[0]);
        case "RESEARCHER_EMP":
            return new ResearcherEmployee(id, name, email, pass, lang, (String) extra[0], (String) extra[1]);
        default:
            throw new IllegalArgumentException("Unknown role: " + role);
        }
    }
}
```

5. Strategy

```
public class ByCitations implements Comparator<ResearchPaper> {
    @Override
    public int compare(ResearchPaper a, ResearchPaper b) {
        return Integer.compare(b.getCitations(), a.getCitations());
    }
}
```

```
public class ByDate implements Comparator<ResearchPaper> {
    @Override
    public int compare(ResearchPaper a, ResearchPaper b) {
        return b.getDate().compareTo(a.getDate());
    }
}
```

```
public class ByPages implements Comparator<ResearchPaper> {
    @Override
    public int compare(ResearchPaper a, ResearchPaper b) {
        return Integer.compare(b.getPages(), a.getPages());
    }
}
```

Also we have class GraduateStudent

```
public class GraduateStudent extends Student implements Researcher {
    private static final long serialVersionUID = 1L;

    public static final int MIN_SUPERVISOR_HINDEX = 3;

    private DegreeType degree;
    private Researcher supervisor;
    private List<ResearchPaper> diplomaPapers = new ArrayList<>();
    private List<ResearchPaper> papers = new ArrayList<>();
    private List<ResearchProject> projects = new ArrayList<>();
```

setSupervisor()method:

```
public void setSupervisor(Researcher r) throws LowHIndexException {
    if (r.calculateHIndex() < MIN_SUPERVISOR_HINDEX) {
        throw new LowHIndexException("Cannot assign " + r.getResearcherName() + " as supervisor. h-index = "
                + r.calculateHIndex() + ", minimum required = " + MIN_SUPERVISOR_HINDEX);
    }
    this.supervisor = r;
    System.out.println(name + "'s supervisor set to " + r.getResearcherName());
}
```

We have class Manager

```
public class Manager extends Employee {
    private static final long serialVersionUID = 1L;

    private ManagerType type;
    private List<Request> complaints = new ArrayList<>();
    private List<Request> employeeRequests = new ArrayList<>();
```

createStatisticalReport() method:

```
public String createStatisticalReport() {
    StringBuilder sb = new StringBuilder();
    sb.append("===== Statistical Report by ").append(name).append(" =====\n");
    Database db = Database.getInstance();
    int sCnt = 0, fCnt = 0;
    double gpaSum = 0;
    for (User u : db.getUsers()) {
        if (u instanceof Student) {
            Student st = (Student) u;
            sCnt++;
            gpaSum += st.getGpa();
            fCnt += st.getFailCount();
        }
    }
    sb.append("Students: ").append(sCnt).append("\n");
    sb.append("Avg GPA: ").append(sCnt == 0 ? "N/A" : String.format("%.2f", gpaSum / sCnt)).append("\n");
    sb.append("Total fails: ").append(fCnt).append("\n");
    sb.append("Total courses: ").append(db.getCourses().size()).append("\n");
    return sb.toString();
}
```

Official Messages & Room Booking
The system supports working official messages about university events. We have class OfficialMessage that stores the event type, content, room, date, and the employee who created it.

```
package models;

import java.io.Serializable;

public class OfficialMessage implements Serializable {
    private static final long serialVersionUID = 1L;

    private String type;
    private String content;
    private String room;
    private Date eventDate;
    private Employee createdBy;

    public OfficialMessage(String type, String content, String room, Date eventDate, Employee createdBy) {
        this.type = type;
        this.content = content;
        this.room = room;
        this.eventDate = eventDate;
        this.createdBy = createdBy;
    }
```

When an exam is planned, a manager calls bookRoomForExam() in Database which automatically creates and broadcasts an official message:

```
public void bookRoomForExam(String room, Date date, Course c, Employee by) {
    OfficialMessage m = new OfficialMessage("EXAM_BOOKING", "Exam for " + c.getName() + " is planned. Room booked.",
            room, date, by);
    addOfficialMessage(m);
}
```

Enumerations

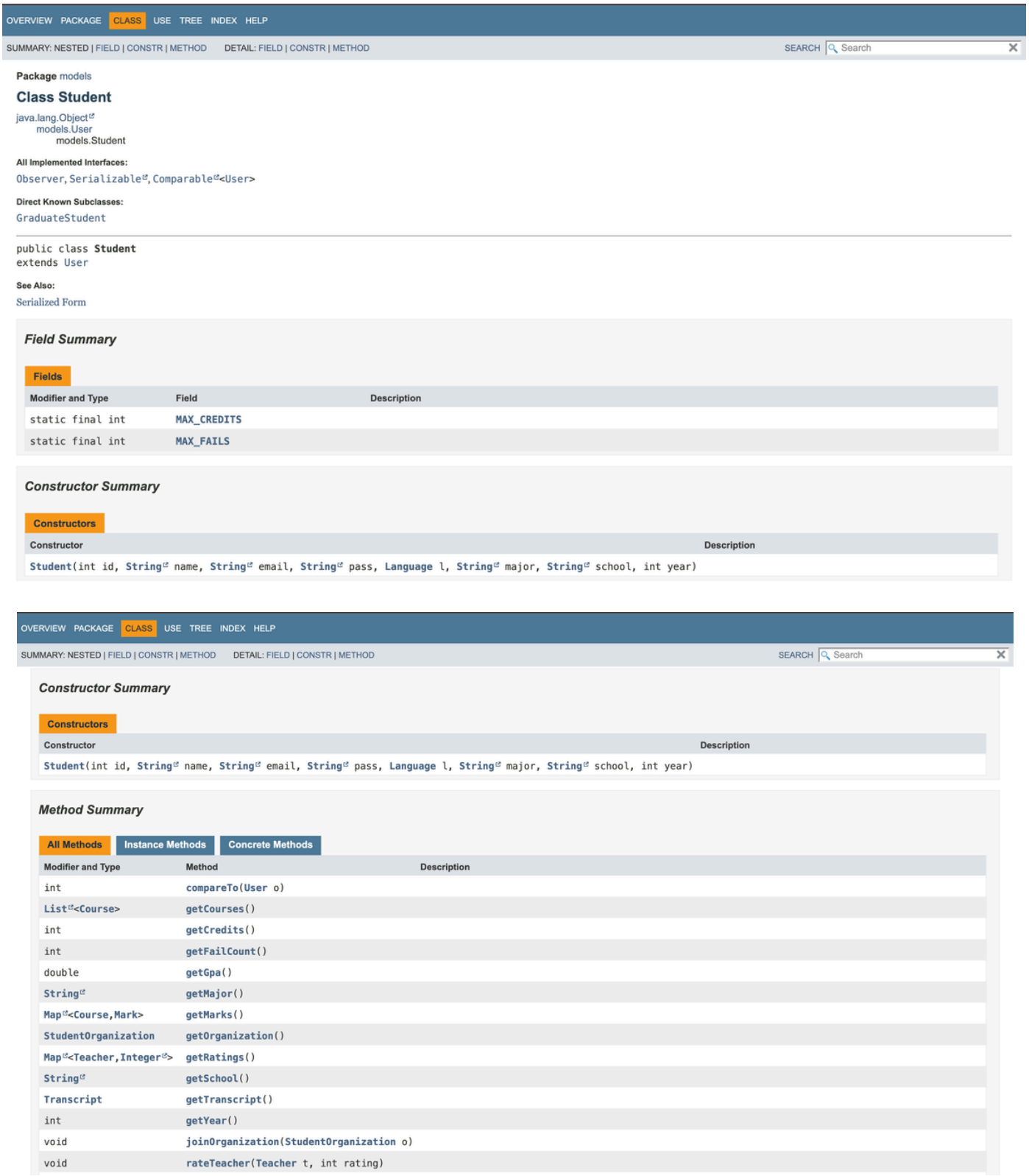
```java
package enums;

public enum TeacherPosition {
    TUTOR, LECTOR, SENIOR_LECTOR, PROFESSOR
}
```

Class News

```java
public class News implements Serializable, Comparable<News> {
    private static final long serialVersionUID = 1L;

    private String title;
    private String content;
    private String topic;
    private boolean isPinned;
    private List<String> comments = new ArrayList<>();
    private Date publishDate;

    public News(String title, String content, String topic) {
        this.title = title;
        this.content = content;
        this.topic = topic;
        this.publishDate = new Date();
    }

    public void addComment(String c) {
        comments.add(c);
    }

    public void pin() {
        this.isPinned = true;
    }
```

The News class implements Serializable and Comparable<News>. It supports comments, pinning (Research topic news are automatically pinned), and sorting by pin status and publish date.

1. Documentation



OVERVIEW PACKAGE CLASS USE TREE INDEX HELP

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD SEARCH Search

Package models

Class Student

java.lang.Object
models.User
models.Student

All Implemented Interfaces:

Observer, Serializable, Comparable<User>

Direct Known Subclasses:

GraduateStudent

public class **Student**
extends User

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
static final int	MAX_CREDITS	
static final int	MAX_FAILS	

Constructor Summary

Constructors

Constructor	Description
Student(int id, String name, String email, String pass, Language l, String major, String school, int year)	

OVERVIEW PACKAGE CLASS USE TREE INDEX HELP

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD SEARCH Search

Constructor Summary

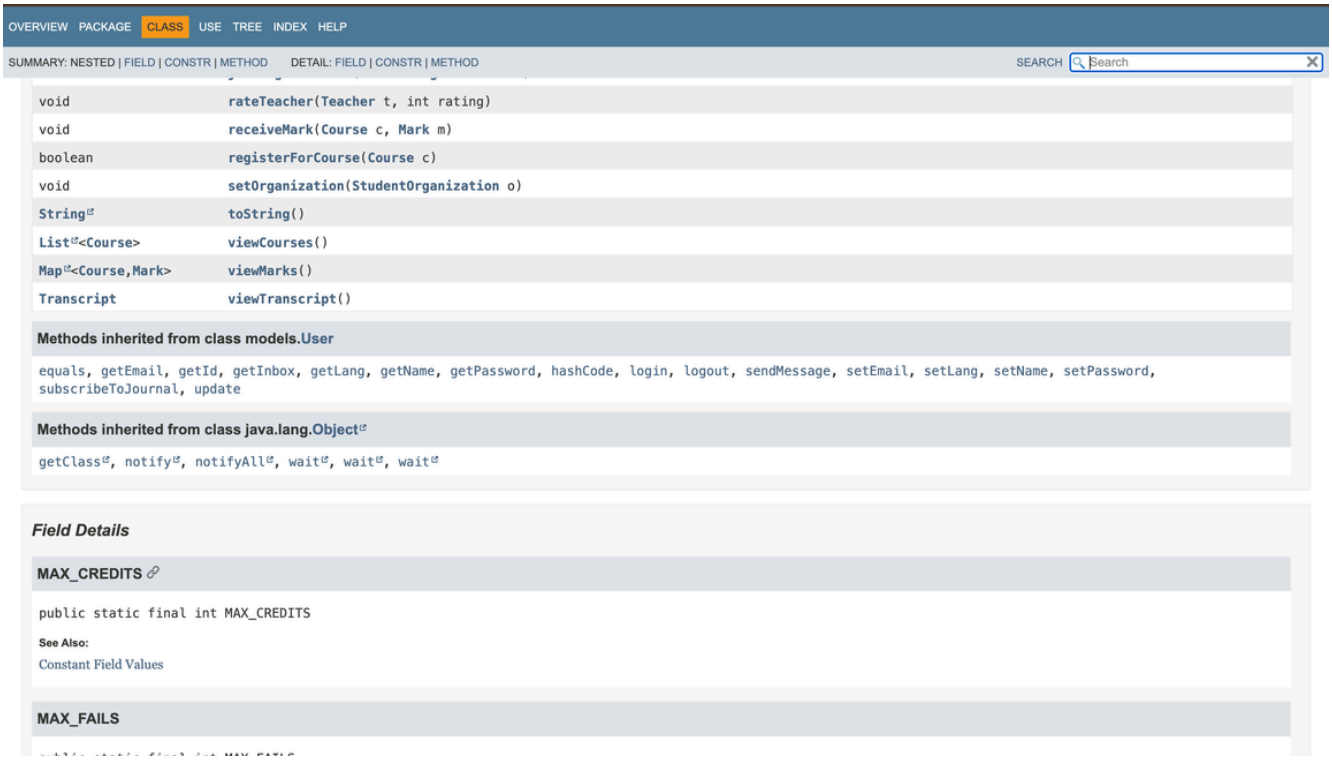
Constructors

Constructor	Description
Student(int id, String name, String email, String pass, Language l, String major, String school, int year)	

Method Summary

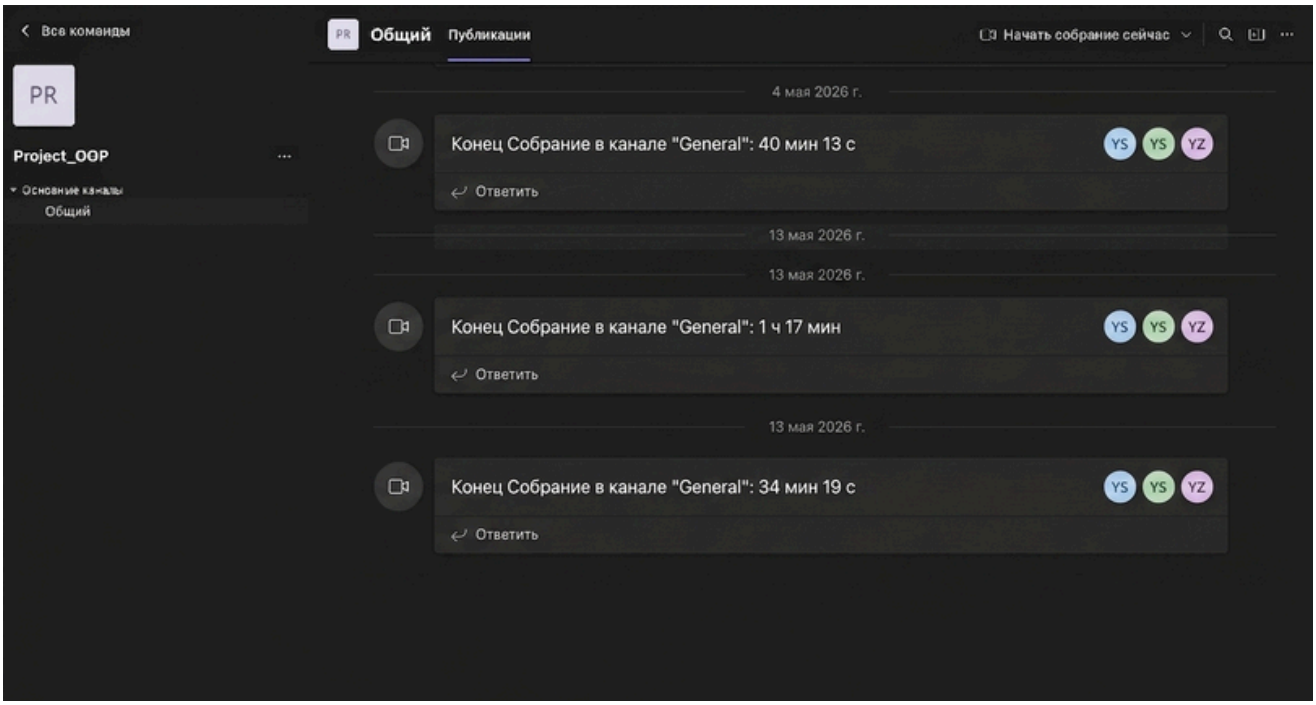
All Methods **Instance Methods** **Concrete Methods**

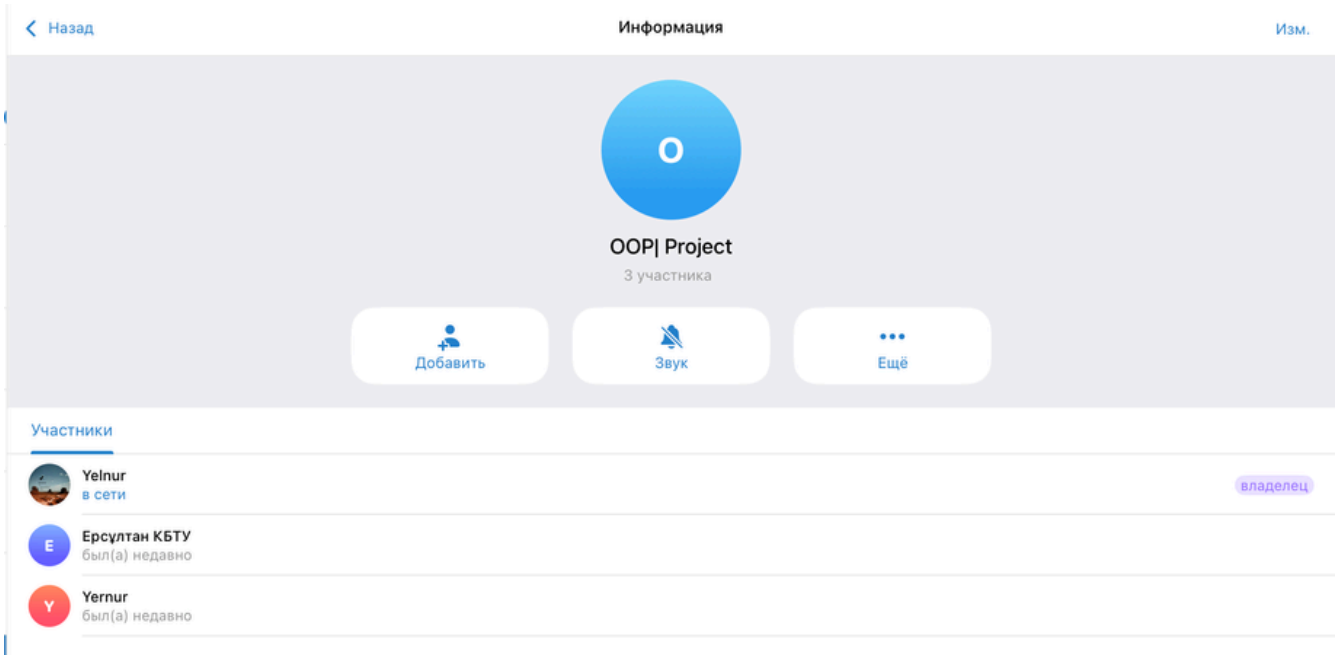
Modifier and Type	Method	Description
int	compareTo(User o)	
List<Course>	getCourses()	
int	getCredits()	
int	getFailCount()	
double	getGpa()	
String	getMajor()	
Map<Course,Mark>	getMarks()	
StudentOrganization	getOrganization()	
Map<Teacher,Integer>	getRatings()	
String	getSchool()	
Transcript	getTranscript()	
int	getYear()	
void	joinOrganization(StudentOrganization o)	
void	rateTeacher(Teacher t, int rating)	



Project Management:

Our team communicated through a Telegram group and a Microsoft Teams channel. At the beginning of the project, we held a general meeting where we divided the work into parts, assigning each team member a specific set of classes and responsibilities. This allowed us to work in parallel and complete the tasks efficiently. Throughout the process, we helped each other and regularly discussed problems and solutions in our group chat.





Problems:

In diagrams: We had difficulties with StarUML/GenMyModel when creating the UML class diagram, as the number of classes was large and it was hard to fit everything clearly. Organizing the relationships between classes such as inheritance, interfaces, and associations took considerable time.

At the second stage, in writing code we faced challenges with implementing design patterns correctly, especially the Observer pattern for journal subscriptions and the Singleton pattern for the Database class. Understanding serialization and making all classes properly Serializable also took extra effort.

At the third stage, generating Javadoc documentation was new to us, but we figured it out completely.